

多态

1.多态基本语法

```
#include<iostream>
using namespace std;

//动态多态满足条件
//1.有继承关系
//2.子类重写父类的虚函数（重写：返回值相同，函数名相同，形参相同）

//动态多态使用
//父类的指针或者引用 指向子类对象 发生多态
class animal{//空类大小为 1 ，加上virtual后 类的大小变为 8，因为编译器生成了一个虚函数指针
public:
    virtual void speak();//这里virtual必写
    {
        cout<<"animal say"<<endl;
    }
};

class cat:public animal{
public:
    void speak();//这里virtual可写可不写
    {
        cout<<"cat say"<<endl;
    }
};

//地制早绑定 在编译阶段确定函数地址
//要想要猫说话，那么这个函数地址就不能提前绑定，需要在运行阶段进行绑定（地址晚绑定）

void dospeak(animal &animal)
{
    animal.speak();
}

void test()
{
    cat cat;
    dospeak(cat);
}

int main()
{
    test();

    return 0;
}
```

案例一

```
#include<iostream>
//#include<string>
using namespace std;

//普通写法
//class calculate{
//public:
//  int getResult(string oper)
//  {
//      if(oper=="+")
//      {
//          return m_num1 + m_num2;
//      }
//      else if(oper=="-")
//      {
//          return m_num1 - m_num2;
//      }
//      else if(oper=="*")
//      {
//          return m_num1 * m_num2;
//      }
//  }
//
//  int m_num1;
//  int m_num2;
//};
//
//void test()
//{
//  calculate c;
//  c.m_num1=30;
//  c.m_num2=20;
//
//  cout<<c.getResult("+")<<endl;
//  cout<<c.getResult("-")<<endl;
//  cout<<c.getResult("*")<<endl;
//}
//
//int main()
//{
//  test();
//
//  return 0;
//}

//多态写法

class basecalculate{
public:
    virtual int getResult()
    {
        return 0;
    }
}
```

```

    int num1;
    int num2;
};

class addcalculate:public basecalculate{
public:
    int getresult()
    {
        return num1 + num2;
    }
};

```

```

class subcalculate:public basecalculate{
public:
    int getresult()
    {
        return num1 - num2;
    }
};

```

```

class mulcalculate:public basecalculate{
public:
    int getresult()
    {
        return num1 * num2;
    }
};

```

//当通过基类指针调用虚函数时，C++ 会根据指针实际指向的派生类对象的类型，来调用该派生类的函数，这就是多态的核心机制

```

void test1()
{
    basecalculate *abc = new addcalculate;
    abc->num1=30;
    abc->num2=20;
    cout<<abc->num1<<"+"<<abc->num2<<"="<<abc->getresult()<<endl;
    delete abc;
}

```

```

void test2()
{
    basecalculate *abc = new subcalculate;
    abc->num1=30;
    abc->num2=20;
    cout<<abc->num1<<"-"<<abc->num2<<"="<<abc->getresult()<<endl;
    delete abc;
}

```

```

void test3()
{
    basecalculate *abc = new mulcalculate;
    abc->num1=30;
    abc->num2=20;
    cout<<abc->num1<<"*"<<abc->num2<<"="<<abc->getresult()<<endl;
}

```

```

        delete abc;
    }

    int main()
    {
        test1();
        test2();
        test3();

        return 0;
    }

```

2. 纯虚函数和抽象类

```

#include<iostream>
using namespace std;

//纯虚函数语法: virtual 返回值类型 函数名 (参数列表) = 0;
//当类中有了纯虚函数, 这个类也被称为抽象类
//抽象类特点: 1. 无法实例化对象 2. 子类必须重写抽象类中的纯虚函数, 否则也属于抽象类

class base{
public:
    virtual void func()=0;//纯虚函数
};

class son:public base{
public:
    void func()
    {
        cout<<"func() 调用"<<endl;
    }
};

void test()
{
    //base b;//抽象类无法实例化对象
    //new base; //抽象类无法实例化对象

    base *base = new son;
    base->func();
}

int main()
{
    test();

    return 0;
}

```

案例二

```
#include<iostream>
using namespace std;

class abstractdrinking{
public:
    virtual void boil()=0;
    virtual void brew()=0;
    virtual void pourincup()=0;
    virtual void putsomething()=0;

    void makedrink()
    {
        boil();
        brew();
        pourincup();
        putsomething();
    }
};

class coffee:public abstractdrinking{
public:
    void boil()
    {
        cout<<"煮农夫山泉"<<endl;
    }
    void brew()
    {
        cout<<"冲泡咖啡"<<endl;
    }
    void pourincup()
    {
        cout<<"倒入杯中"<<endl;
    }
    void putsomething()
    {
        cout<<"加入糖和牛奶"<<endl;
    }
};

class tea:public abstractdrinking{
public:
    void boil()
    {
        cout<<"煮矿泉水"<<endl;
    }
    void brew()
    {
        cout<<"冲泡茶叶"<<endl;
    }
    void pourincup()
    {
        cout<<"倒入杯中"<<endl;
    }
}
```

```

    void putsomething()
    {
        cout<<"加入枸杞"<<endl;
    }
};

void dowork(abstractdrinking *abs)
{
    abs->makedrink();
    delete abs;
}

void test()
{
    dowork(new coffee);
    cout<<"-----"<<endl;
    dowork(new tea);
}

int main()
{
    test();

    return 0;
}

```

3.虚析构与纯虚析构

```

#include<iostream>
using namespace std;

class animal{
public:
    animal()
    {
        cout<<"animal 构造函数调用"<<endl;
    }

    virtual void speak()=0;

    //利用虚析构或者纯虚析构都可以解决 父类指针释放子类对象时不干净的问题
    // virtual ~animal()
    // {
    //     cout<<"animal 析构函数调用"<<endl;
    // }

    //纯虚析构
    //如果类中有纯虚析构，则该类属于抽象类，无法实例化对象
    //纯虚析构既要在类中声明，又要在类外具体实现
    virtual ~animal()=0;
};

animal::~~animal()
{

```

```

        cout<<"animal 纯虚析构"<<endl;
    }

class cat:public animal{
public:
    cat(string name)
    {
        cout<<"cat 构造函数调用"<<endl;
        m_name=new string(name);//如果子类中没有堆区数据，可以不写 虚析构或者纯虚析构
    }

    void speak()
    {
        cout<<*m_name<<" "<<"cat say"<<endl;
    }

    ~cat()
    {
        if(m_name!=NULL)
        {
            cout<<"cat 析构函数调用"<<endl;
            delete m_name;
            m_name=NULL;
        }
    }

    string *m_name;
};

void test()
{
    animal *animal=new cat("Tom");//当使用new关键字申请内存时，会自动调用构造函数，并将
new关键字括号里的值传递给构造函数
    animal->speak();
    delete animal;//利用虚析构可以解决 利用父类指针释放子类对象的问题
}

int main()
{
    test();

    return 0;
}

```

案例三

```

#include<iostream>
using namespace std;

class cpu{
public:
    virtual void calculate()=0;
};

class vediocard{

```

```

public:
    virtual void display()=0;
};

class memory{
public:
    virtual void storage()=0;
};

class computer{
public:
    computer(cpu *cpu,vediocard *vc,memory *mem)
    {
        m_cpu=cpu;
        m_vc=vc;
        m_mem=mem;
    }

    //提供工作函数
    void work()
    {
        m_cpu->calculate();
        m_vc->display();
        m_mem->storage();
    }

    //提供析构函数，释放三个电脑零件
    ~computer()
    {
        if(m_cpu!=0)
        {
            delete m_cpu;
            m_cpu=NULL;
        }
        if(m_vc!=0)
        {
            delete m_vc;
            m_vc=NULL;
        }
        if(m_mem!=0)
        {
            delete m_mem;
            m_mem=NULL;
        }
    }

private:
    cpu *m_cpu;
    vediocard *m_vc;
    memory *m_mem;
};

```

```

//intel厂商的组件
class intelcpu:public cpu{

```



```

public:
    void calculate()
    {
        cout<<"intel cpu work"<<endl;
    }
};

class intelvediocard:public vediocard{
public:
    void display()
    {
        cout<<"intel vediocard work"<<endl;
    }
};

class intelmemory:public memory{
public:
    void storage()
    {
        cout<<"intel storage work"<<endl;
    }
};

//lenovo厂商的组件
class lenovocpu:public cpu{
public:
    void calculate()
    {
        cout<<"lenovo cpu work"<<endl;
    }
};

class lenovovediocard:public vediocard{
public:
    void display()
    {
        cout<<"lenovo vediocard work"<<endl;
    }
};

class lenovomemory:public memory{
public:
    void storage()
    {
        cout<<"lenovo storage work"<<endl;
    }
};

void test()
{
    //第一台电脑零件
    cpu *p_intelcpu=new intelcpu;
    vediocard *p_intelcard=new intelvediocard;
    memory *p_intelmemory=new intelmemory;
}

```

```
//创建第一台电脑
//new返回一个computer类型的指针，所以要用一个computer类型的指针接收
//当使用new关键字申请内存时，会自动调用构造函数，并将new关键字括号里的值传递给构造函数
computer *computer1=new computer(p_intelcpu,p_intelcard,p_intelmemory);
computer1->work();
delete computer1;

//创建第二台电脑
computer *computer2=new computer(new lenovocpu,new lenovovediocard,new
lenovomemory);
computer2->work();
delete computer2;

//创建第三台电脑
computer *computer3=new computer(new intelcpu,new lenovovediocard,new
lenovomemory);
computer2->work();
delete computer3;
}

int main()
{
    test();

    return 0;
}
```